

PES University — Department of Computer Science & Engineering

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Problem Statement #42: Internal Microservice Catalog & Health Portal | Student: Subramani B M | SRN: PES1UG24CS473 | Section: H

1. Epics (Mapped from Lab 1 Functional Requirements)

#	Epic Name	FR Mapped	Description	Total SP	Stories
1	Service Catalog Management	FR-002	Register, update, search and filter microservices in a centralized catalog with metadata validation.	13	4
2	Health Monitoring & Availability	FR-001	Automated 30-second interval health probing with rolling 24-hour availability calculation.	13	4
3	Dependency Graph Visualization	FR-003	Interactive upstream/downstream dependency graph with auto-discovery and click-to-navigate.	8	3
4	Alerting & Notification System	FR-004	Multi-channel downtime alerts (Email, Slack, Webhook) triggered on 3 consecutive failed probes.	8	3
5	API Documentation Aggregator	FR-005	Aggregate and display auto-generated OpenAPI/Swagger documentation per microservice.	5	2
Grand Total				47	16

2. User Stories with Priority & Story Points

Story ID	Epic	User Story (As a... I want... So that...)	Priority	SP	Sprint
US-1.1	Service Catalog Mgmt	As a DevOps Engineer, I want to register a new microservice by providing its name, owner team, version, and repository URL, so that it appears in the centralized catalog.	High	5	Sprint 1
US-1.2	Service Catalog Mgmt	As a DevOps Engineer, I want to update existing microservice metadata (version, environment, owner), so that the catalog reflects the current state.	High	3	Sprint 1
US-1.3	Service Catalog Mgmt	As a System Architect, I want to search and filter the service catalog by name, team, environment, or tech stack, so that I can quickly find relevant services.	High	3	Sprint 1
US-1.4	Service Catalog Mgmt	As a DevOps Engineer, I want the system to validate metadata completeness upon registration, so that incomplete entries are rejected.	Medium	2	Sprint 2
US-2.1	Health Monitoring	As a DevOps Engineer, I want the system to automatically ping health endpoints every 30 seconds, so that I have continuous visibility into service status.	High	5	Sprint 1
US-2.2	Health Monitoring	As a DevOps Engineer, I want to view a real-time health dashboard with live status, timestamps, and response times, so that I can instantly assess system health.	High	3	Sprint 1
US-2.3	Health Monitoring	As a DevOps Engineer, I want the system to calculate rolling 24-hour availability percentages, so that I can track reliability and meet SLA targets.	Medium	3	Sprint 2
US-2.4	Health Monitoring	As a DevOps Engineer, I want to configure custom health check intervals and timeout thresholds per microservice, so that monitoring adapts to each service.	Low	2	Sprint 2
US-3.1	Dependency Graph	As a System Architect, I want to view an interactive dependency graph showing upstream/downstream relationships, so that I understand the system topology.	High	5	Sprint 1
US-3.2	Dependency Graph	As a System Architect, I want to click on a graph node to navigate to that service's detail page, so that I can drill down from the graph view.	Medium	2	Sprint 2
US-3.3	Dependency Graph	As a System Architect, I want the graph to auto-discover new dependencies within 2 minutes, so that the topology stays current.	Low	1	Sprint 2
US-4.1	Alerting & Notifications	As a DevOps Engineer, I want the system to dispatch downtime alerts via Email, Slack, and Webhook when 3 consecutive probes fail, so that on-call personnel are notified.	High	5	Sprint 1 + 2
US-4.2	Alerting & Notifications	As a DevOps Engineer, I want to configure notification channels and on-call contacts per microservice, so that alerts reach the right team.	Medium	2	Sprint 2
US-4.3	Alerting & Notifications	As a DevOps Engineer, I want to receive a recovery notification when a down service becomes healthy, so that I know the incident is resolved.	Low	1	Sprint 2
US-5.1	API Documentation	As a Developer, I want to browse aggregated API documentation (endpoints, schemas, payloads) per microservice, so that I can integrate without scattered docs.	High	3	Sprint 2
US-5.2	API Documentation	As a DevOps Engineer, I want to upload or link an OpenAPI/Swagger spec during registration, so that API docs are auto-generated within 2 minutes.	Medium	2	Sprint 2

3. Sprint Summary

Sprint	Duration	Stories Included	Total SP
Sprint 1	1 week	US-1.1, US-1.2, US-1.3, US-2.1, US-2.2, US-3.1, US-4.1 (partial)	26
Sprint 2	1 week	US-4.1 (cont.), US-5.1, US-2.3, US-1.4, US-4.2, US-5.2, US-3.2, US-2.4, US-4.3, US-3.3	21

Reflection Questions & Answers

Q1: Did your estimations reflect the actual effort? Yes, the Fibonacci-based story point estimates largely aligned with the simulated effort. High-complexity stories (US-1.1, US-2.1, US-3.1, US-4.1) at 5 points appropriately reflected their architectural significance and multi-component integration requirements — these stories required designing database schemas, implementing API endpoints, and building frontend components. Supporting stories at 1–3 points matched their lower complexity, involving configuration or incremental feature additions. Sprint 1 carried 26 points compared to Sprint 2's 21 points, which in a real project would require velocity calibration after Sprint 1 to better balance capacity across iterations.
Q2: Was your backlog well-prioritized? Yes. The backlog was structured following the MoSCoW (Must-have, Should-have, Could-have) prioritization principle. High-priority stories delivering core user value — service registration (US-1.1), health monitoring (US-2.1), dependency visualization (US-3.1), and downtime alerting (US-4.1) — were scheduled for Sprint 1, ensuring the minimum viable product was delivered first. Medium and Low-priority enhancement stories (custom intervals, recovery notifications, auto-discovery) were deferred to Sprint 2. This ensures the product is usable after Sprint 1, even if Sprint 2 encounters delays.
Q3: How did your simulated sprint align with your plan? The simulated sprint closely followed the plan. All High-priority stories in Sprint 1 were moved sequentially from To Do → In Progress → Done, reflecting realistic task progression where dependent stories (e.g., catalog registration before search) were completed first. Sprint 2 completed the remaining backlog items without scope creep. Both sprints were completed within the 1-week timeboxed duration, demonstrating that the story point estimation and sprint capacity planning were well-calibrated for the team's velocity.
Q4: What insights did the burndown chart give about your team's capacity? The burndown chart revealed a steady, near-linear decline in remaining story points across both sprints, indicating well-distributed workload and consistent team velocity. Sprint 1's guideline (grey line) and actual remaining work (red line) tracked closely, suggesting accurate estimation. The slight difference between Sprint 1 (26 points) and Sprint 2 (21 points) totals reflects adaptive planning — using Sprint 1's actual velocity as input for Sprint 2's capacity, which is a core Agile principle. The chart confirmed no major estimation outliers or blockers occurred during execution.